

Numerische Methoden – Aufgabenblatt

Alle Aufgaben aus dem Vorlesungsskript “*Numerical Methods*” (Prof. Dr.-Ing. M. Schutera) – 1:1 extrahiert, mit Platz zum Ausfüllen.

Name: _____

Datum: _____

Kapitel 1 – Einstieg in die Numerischen Methoden

1.1 – Minimum der Sinusfunktion [Handrechnung]

Bestimmen Sie das Minimum von $\sin(x)$ auf $[-3, 1]$, indem Sie die Funktion mit Schrittweite h diskretisieren, an den Gitterpunkten auswerten und den kleinsten Wert auswählen. Vergleichen Sie mit dem analytisch exakten Minimum.



1.2 – Grid-Search [Handrechnung]

Führen Sie eine Grid-Search für w, b mit Schrittweite 2,5 über $\{0; 2,5; 5; 7,5; 10\}$ durch (System $\begin{pmatrix} 2 & 1 \\ 5 & 1 \end{pmatrix} \begin{pmatrix} w \\ b \end{pmatrix} = \begin{pmatrix} 11 \\ 13 \end{pmatrix}$). Fehler $\text{error}_1 = |2w + b - 11|$, $\text{error}_2 = |5w + b - 13|$. Wählen Sie das Paar (w, b) mit kleinstem akkumuliertem Fehler.



Setzen Sie die Grid-Search im vorgehosteten Jupyter-Notebook (Python-Vorlage) um. Nutzen Sie die Selbstreflexionsfragen als Leitfaden.

Denkanstoß. Fällt Ihnen eine einfache Möglichkeit ein, das Verhältnis von Genauigkeit zu Rechenaufwand der Grid-Search zu verbessern?

Selbstreflexionsfragen

- Warum ist die Wahl der Schrittweite h wichtig (Genauigkeit vs. Rechenzeit)?
- Wie beeinflusst das Intervall $[a, b]$ Ergebnisse und Lage der Extrema?
- Unterschiede stetige vs. diskretisierte Funktion – Implikationen?



2.1 – Rundungsfehler [Handrechnung]

Betrachten Sie 10/3. Schreiben Sie die Dezimalentwicklung aus und bestimmen Sie den Rundungsfehler beim Abschneiden nach drei Nachkommastellen.

2.2 – 2-Bit-Mantisse & Maschinenepsilon [Handrechnung]

Eine unnormalisierte Mantisse hat 2 Bit ($0.xx_2$). Bestimmen Sie alle darstellbaren Werte und $\varepsilon_{\text{mach}} = 2^{-t}$.
Relative vs. absolute Präzision?

A large grid of graph paper, consisting of 20 columns and 30 rows of small squares, intended for hand calculations.

2.3 – Auslöschung von Hand [Handrechnung]

Zwei Zahlen, 8 signifikante Stellen. **(a)** $a = 12345678,5$, $b = 12345678,0$: $\tilde{a}, \tilde{b}$, $\tilde{a} - \tilde{b}$ vs. $a - b$. **(b)** $a = 12345678,4$, $b = 12345678,0$: ebenso. Beobachtung beim relativen Fehler?

A large grid of graph paper, consisting of 30 columns and 30 rows of small squares, intended for hand calculations.

2.4 – Festkomma-Skalierung [Handrechnung]

Bereich $[-1000, 1000]$, 32-Bit-Integer, Skalierungsfaktor 10^6 . Wie wird 1,234567 gespeichert? Präzision (kleinste Differenz) und maximaler Rundungsfehler?

A large grid of graph paper, consisting of 30 columns and 30 rows of small squares, intended for hand calculations.

2.5 – Maschinenepsilon (hands on) [\[Code\]](#)

Bestimmen Sie das Maschinenepsilon eines Systems per Pseudocode. Festkomma vs. Gleitkomma? Welcher Datentyp für einen Taschenrechner und warum?

2.6 – Auslöschung demonstrieren (hands on) [\[Code\]](#)

Wie demonstrieren Sie Auslöschung am Rechner? Schreiben Sie Pseudocode auf, bevor Sie weiterlesen.

2.7 – Argumentation [Konzept]

Wie leistet $f(\epsilon) = \frac{1}{a + \epsilon - b}$ für kleines ϵ und $a = b$ dies? Was erwarten Sie für sehr kleines ϵ ?

Randnotiz: Was passiert für $a > b$? Denken Sie in signifikanten Stellen.



Denkanstoß. Fallen Ihnen Metriken für numerische Verfahren ein, basierend auf den besprochenen Approximationen und Fehlern?

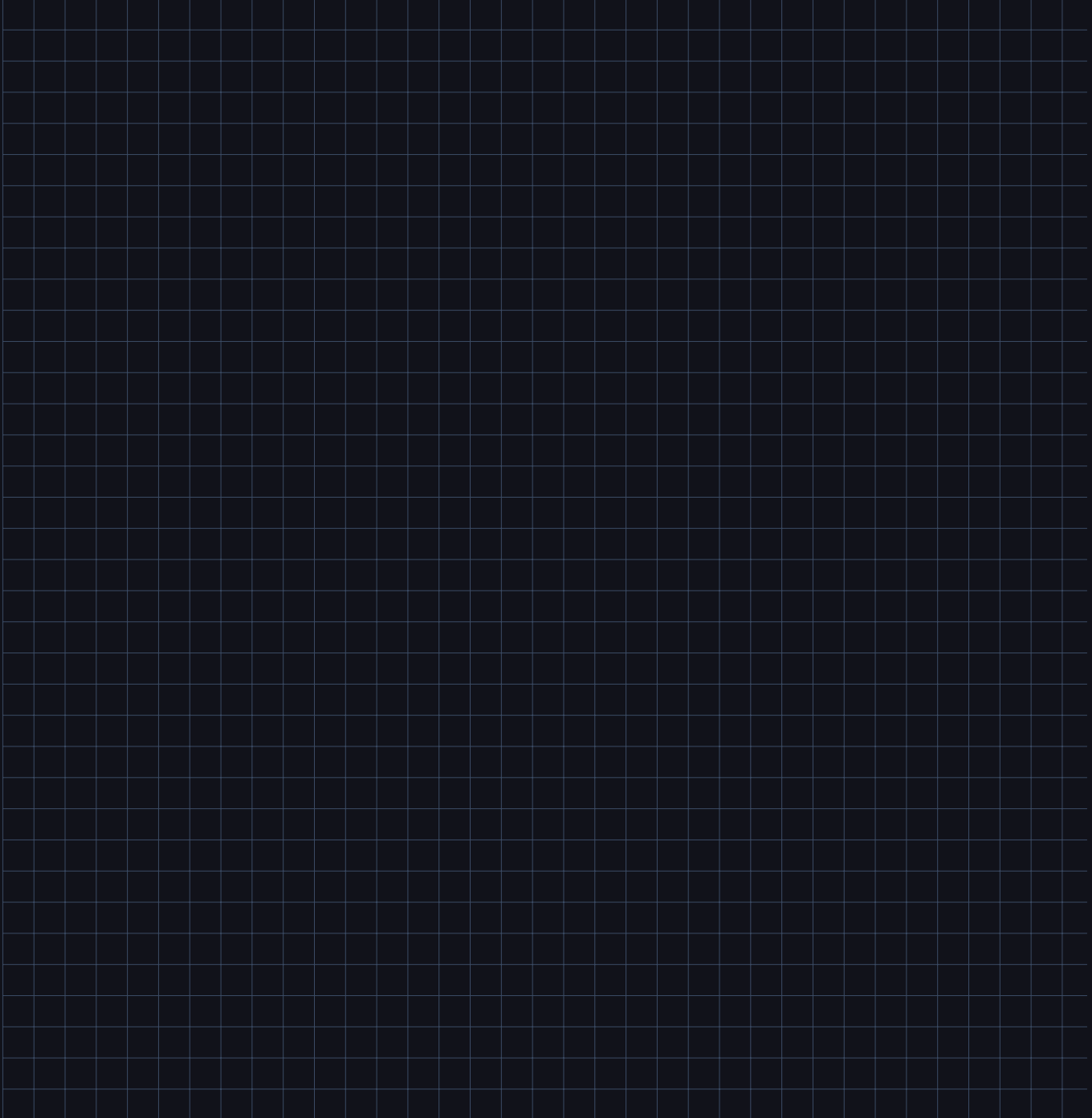


Selbstreflexionsfragen

- Aufbau und Komponenten der Gleitkommadarstellung?
- Was ist das Maschinenepsilon, Bezug zur Genauigkeit?
- Praktische Auswirkungen auf numerische Berechnungen?

3.1 – Minimum der Sinusfunktion (Eigenschaften) [Handrechnung]

Minimum von $\sin(x)$ auf $[-3, 1]$ erneut; diskretisiert mit $h = 1$ und $h = 0,2$; Eingabe gestört $\tilde{x} = x \pm 0,25$. Analysieren und quantifizieren Sie **Kondition**, **Stabilität**, **Konsistenz** und **Konvergenz** in beiden Fällen mit den Formeln aus dem vorigen Abschnitt.



3.2 – Stabilität und Kondition [Beweis]

Zeigen Sie: Die Stabilität entspricht der Kondition des Systems für $h \rightarrow 0$.



3.3 – Compute-getriebene Fehleranalyse (hands on) [\[Code\]](#)

Brute-forcen Sie die Fehleranalyse beim Lösen des Zwei-Gleichungs-Systems aus Kap. 1; bestimmen und optimieren Sie Kondition/Stabilität/Konsistenz/Konvergenz.



Selbstreflexionsfragen

- Konditionierung des Zwei-Gleichungs-Systems (vgl. $\min \sin$)?
- Konvergiert die Stabilität mit kleineren Schrittweiten? Wogegen?
- Wirkung von Eingabestörungen; Zusammenspiel mit h ?
- Konvergenz vs. Stabilität & Konsistenz (Bias/Varianz)?
- Genauigkeitsgrenze für immer kleinere h ? Warum?
- Welches weitere Problem beim Verfeinern von h ?



Kapitel 4 – Newton-Verfahren

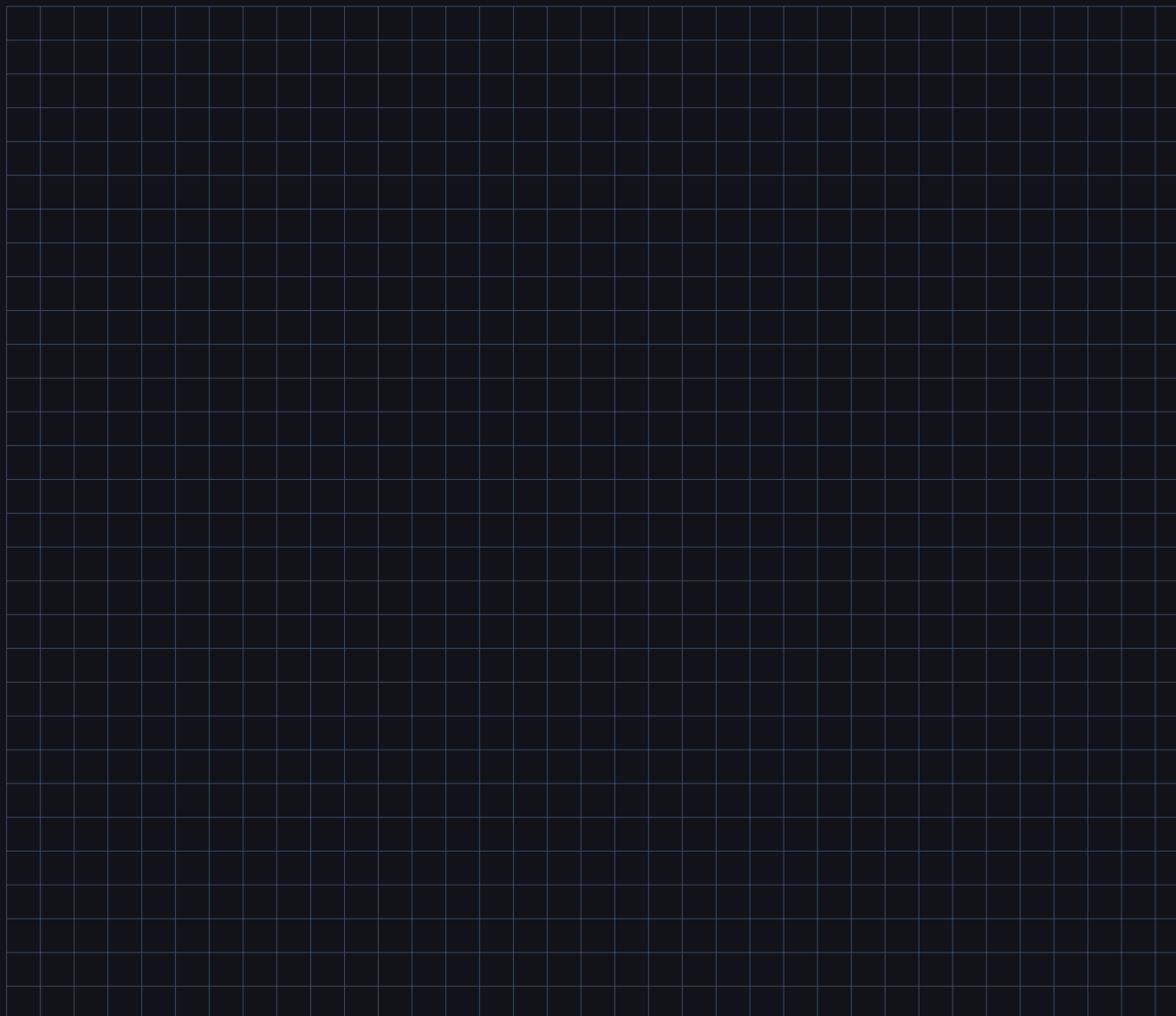
4.1 – Newton für $\sqrt{2}$ [Handrechnung]

Berechnen Sie $\sqrt{2}$ mit Newton ($f(x) = x^2 - 2$), Start $x_0 = 2$. Einige Iterationen; beobachten Sie das Quadrieren des Fehlers.



4.2 – Newton vs. Grid-Search für $\sin(x) = 0$ [Handrechnung]

Nullstelle von $\sin(x) = 0$ auf $[2,5; 4]$ (nahe π). **(a)** Grid-Search, $h = 0,3$. **(b)** Newton ($f' = \cos$), $x_0 = 3,5$. **(c)** Genauigkeit & Auswertungen vergleichen.



4.3 – Denksportaufgabe: Versagensmodi [Schaubild]

Visualisieren Sie die Versagensmodi des Newton-Verfahrens im Schaubild von $f(x) = x^3 - x$.



4.4 – Newton von Hand [Handrechnung]

Nullstelle von $f(x) = x^3 - x = 0$ (Nullstellen $-1, 0, 1$), erst geometrisch, dann von Hand; gesucht $x = 1$ ab $x_0 = 1,4$.



4.5 – Sekantenverfahren von Hand [Handrechnung]

Sekantenverfahren auf $f(x) = x^3 - x$ mit $x_0 = 1,2$, $x_1 = 1,4$; erst geometrisch, dann von Hand.



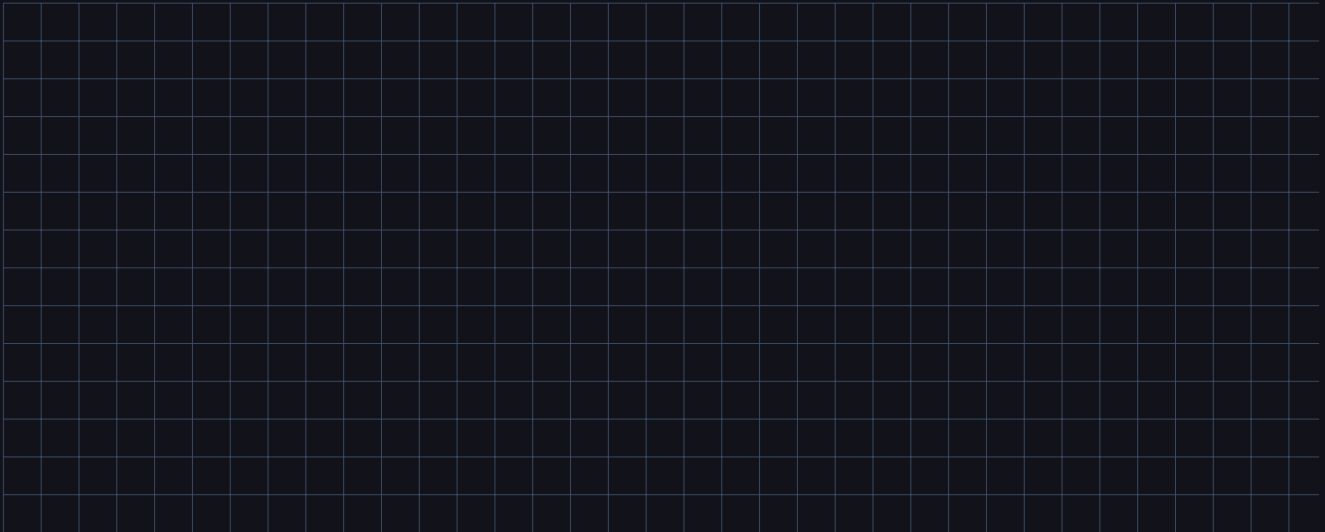
4.6 – Geometrische Fehlersuche [Handrechnung]

Startpunkte x_0 für drei Versagensmodi: verschwindende Ableitung; Konvergenz gegen eine Nicht-Ziel-Nullstelle; Oszillation zwischen Punkten.



4.7 – Enter the machine [\[Code\]](#)

Konvergenz(-raten) von Grid-Search, Newton, Sekanten in Python; Versagensmodi über x_0 erkunden (geometrisch, analytisch, im Code).



Selbstreflexionsfragen

- Hauptvorteil von Newton gegenüber Grid-Search?
- Warum genügt lineare Approximation (Taylor)?
- Mächtigkeit quadratischer Konvergenz ($10^{-1} \rightarrow 10^{-16}$?).
- Wann Sekanten vorziehen, wann nicht?

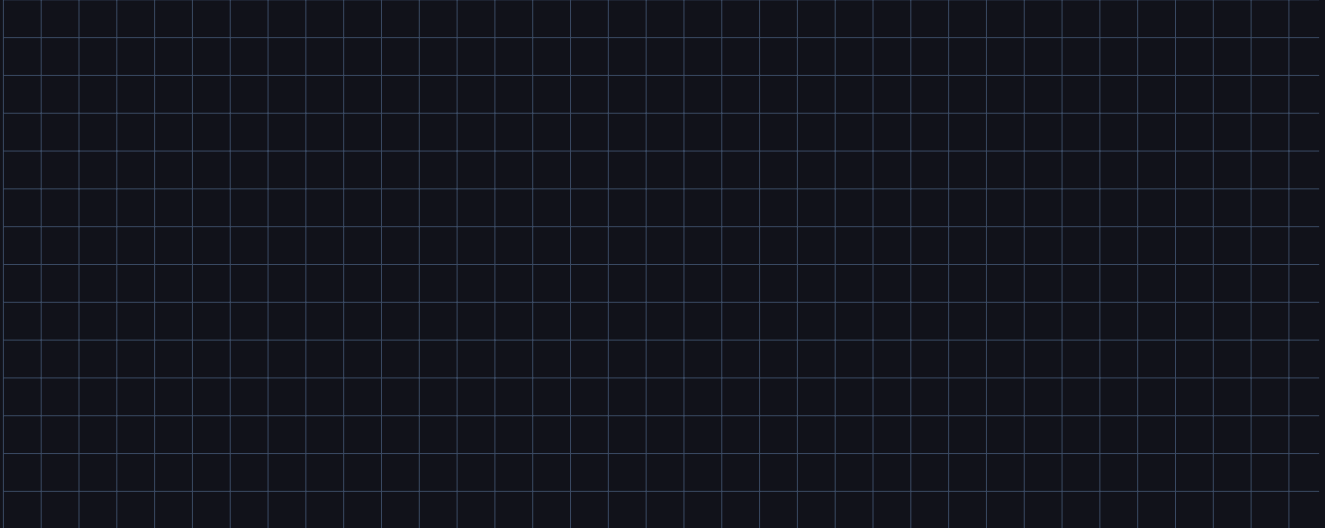


Kapitel 5 – Globale Optimierung

5.1 – Newton auf Shekel, weiterer Startpunkt [\[Code\]](#)

Newton auf der 1D-Shekel-Funktion von verschiedenen Startpunkten; wohin konvergiert es? Diskutieren Sie das allgemeine Verhalten.

Randnotiz: Wie ändern Sie die Update-Regel, um stattdessen Maxima zu finden?



5.2 – Random Restarts [Handrechnung]

5 lokale Minima, globales Becken $p = 0,15$. P bei $K = 10$ ($P = 1 - (1 - p)^K$); K für 99% ($K = \frac{\ln(1-P)}{\ln(1-p)}$); dann K für $p = 0,05$ und $p = 0,01$.



5.3 – Eigene 1D-Shekel-Funktion [\[Code\]](#)

Eigene 1D-Shekel-Funktion entwerfen; lokale Verfahren anwenden, Newton-Versagen zeigen; verschiedene x_0 untersuchen.



5.4 – Die Suche zu Minima lenken [\[Konzept + Code\]](#)

$|f''|$ -Trick: Vorzeichen der Krümmung im Nenner umkehren $\rightarrow$ Schritt geht stets bergab. Geometrische Bedeutung? Dann im Code testen.



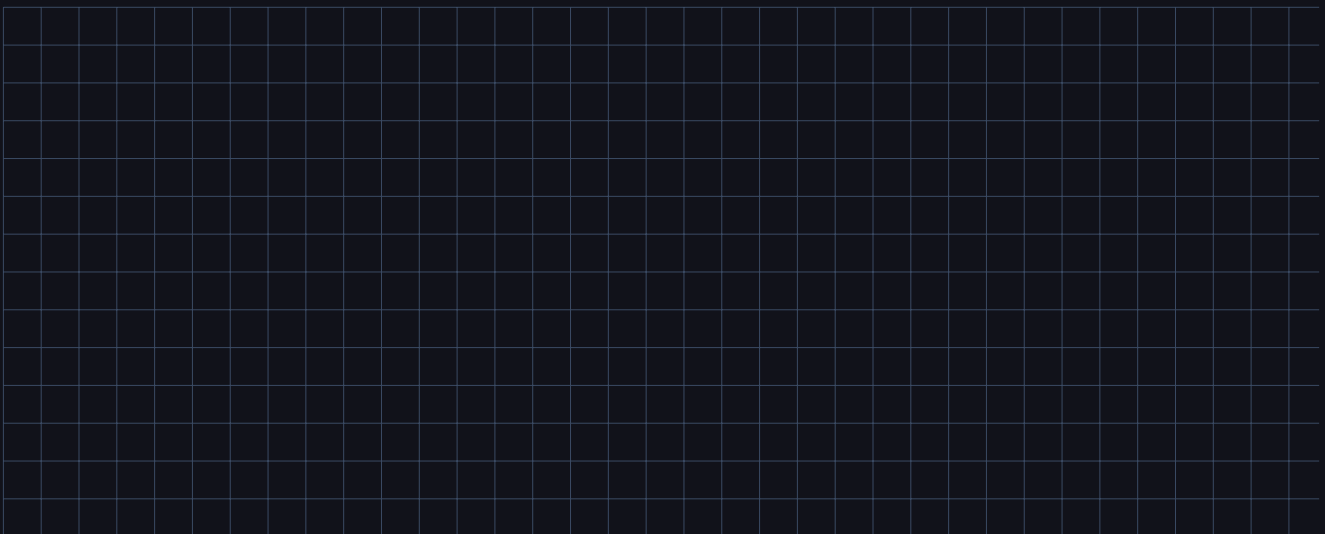
5.5 – Einzugsgebiet (Basin of Attraction) [Schaubild]

Markieren Sie in den Shekel-Plots die Einzugsgebiete jedes lokalen Minimums und schätzen Sie ihre relativen Größen. Welches ist global? Bezug zur Random-Restart-Wahrscheinlichkeit?



5.6 – Lokalen Minima entkommen [Code]

Random Restarts und Basin Hopping auf Ihrer Shekel-Funktion; Funktionsiterationen bis zum globalen Minimum vergleichen. Einfluss der Schrittweitenverteilung?



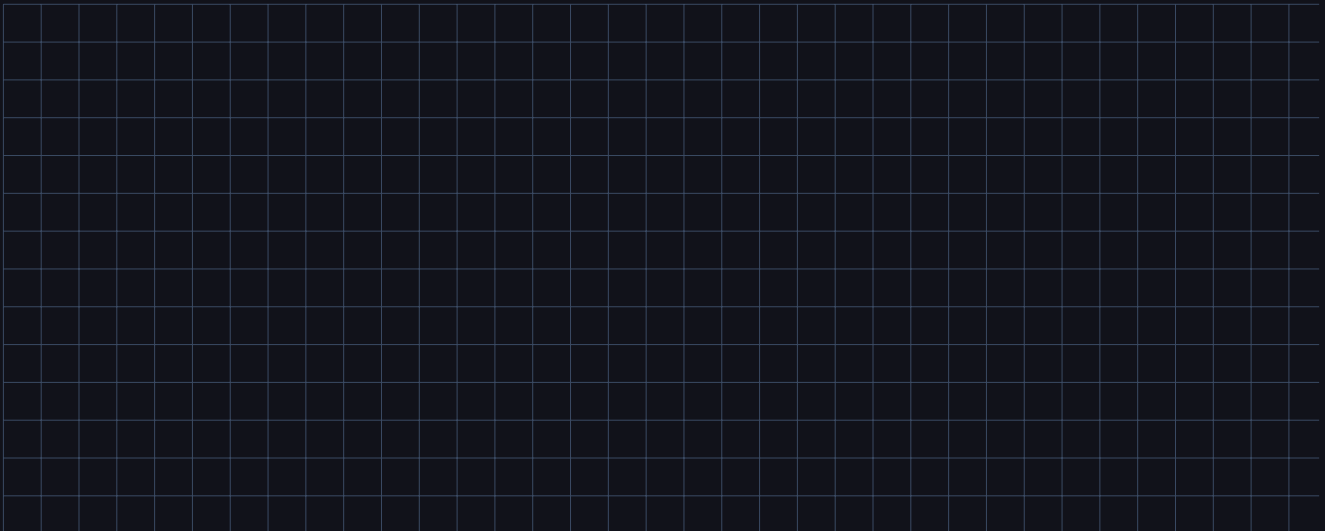
5.7 – Nicht-konvexe Becken [Konzept]

Notieren Sie eine Funktion, bei der die Einzugsgebiete nicht leicht abzugrenzen sind.



5.8 – Adaptive Schrittweite [Konzept + Code]

Geschickter Weg, die Schrittweitenverteilung automatisch anzupassen? Gute Heuristik? Wie katastrophale Sprünge verhindern? Implementieren.



5.9 – Noise and landscape engineering [Konzept]

Wie wirkt das Approximieren der Loss-Landschaft auf Mini-Batches (unterschiedliche Punktauswahl)?



5.10 – Code-Übung: Restarts vs. Basin Hopping [\[Code\]](#)

Random Restarts und Basin Hopping auf der 1D-Shekel-Funktion implementieren; Funktionsauswertungen bis zum globalen Minimum bei $x \approx 4$ vergleichen.



5.11 – Einzugsgebiete kartieren [Handrechnung]

Lokaler Optimierer $\rightarrow$ nächstes Foxhole. **(a)** Einzugsgebiete der Foxholes bei $x = 0, 4, 7$ skizzieren – Basin-Grenzen? **(b)** Sprung $\delta \sim \text{Uniform}(-3, 3)$ ab $x^\circ = 7$: $P(\text{im globalen Becken})$? **(c)** Warum Basin Hopping hier evtl. schneller als Random Restarts?



Selbstreflexionsfragen

- Warum versagt lokale Optimierung auf nicht-konvexen Landschaften?
- Aufwand vs. Größe des globalen Beckens (Random Restarts)?
- Basin Hopping vs. Random Restarts?
- Was sagt $f''(x_n)$ über die Loss-Landschaft, wie nutzbar machen?
- Wie helfen verrauschte (Mini-Batch-)Schätzungen beim Entkommen?
- x_0 in einem lokalen Becken: Wege zum Entkommen – Wirksamkeit?

Denkanstoß. Was, wenn wir einer hochfrequenten Version von Shekels Foxholes gegenüberstehen? Welches Problem tritt auf?

Kapitel 6 – Numerische Integration

6.1 – Exakter Wert [Handrechnung]

Berechnen Sie $\int_{-2}^{-1} \sin(x) dx$ exakt (Referenzwert).



6.2 – Mittelpunktsregel von Hand [Handrechnung]

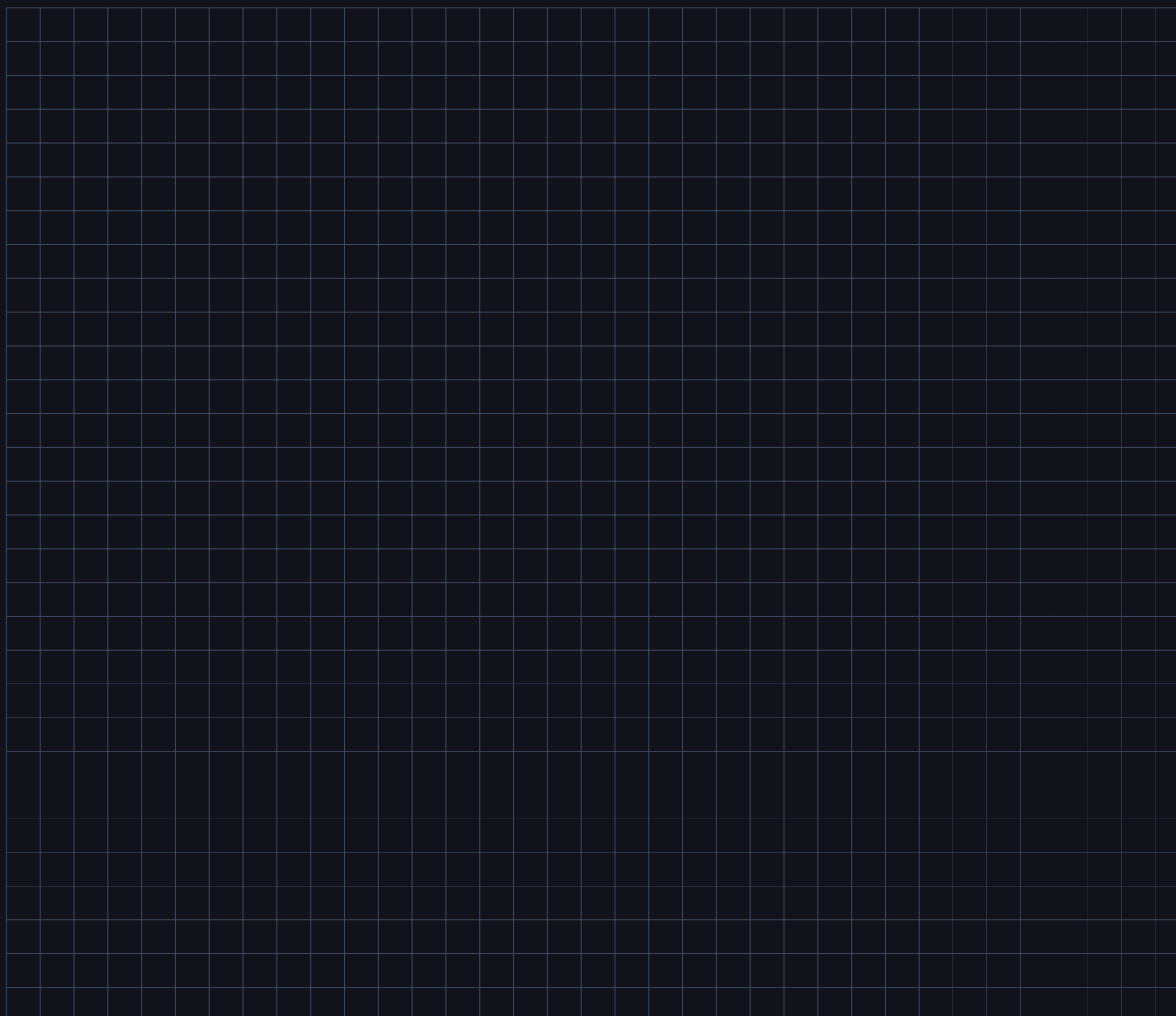
$\int_{-2}^{-1} \sin(x) dx$ mit der Mittelpunktsregel, $n = 1$.



6.3 – Mittelpunktsregel mit mehr Intervallen [Handrechnung]

Mittelpunktsregel mit $n = 2$ (Mittelpunkte $-1,75$ und $-1,25$).

Randnotiz: Berechnen Sie die Approximation auch an den Endpunkten und vergleichen Sie.



6.4 – Trapezregel von Hand [Handrechnung]

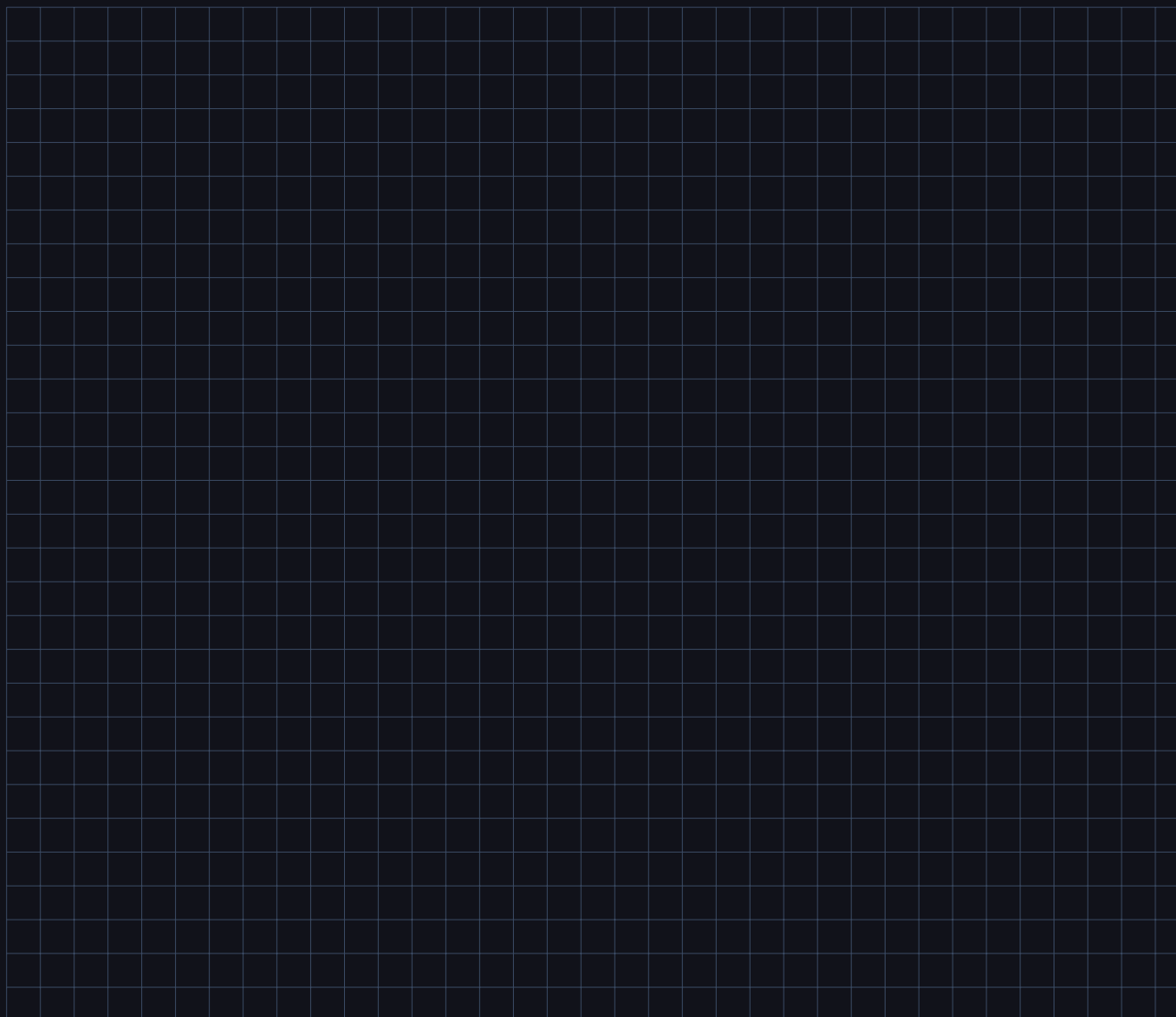
Trapezregel, $n = 2$; Stützstellen $-2, -1, 5, -1$.



6.5 – Simpson-Regel von Hand [Handrechnung]

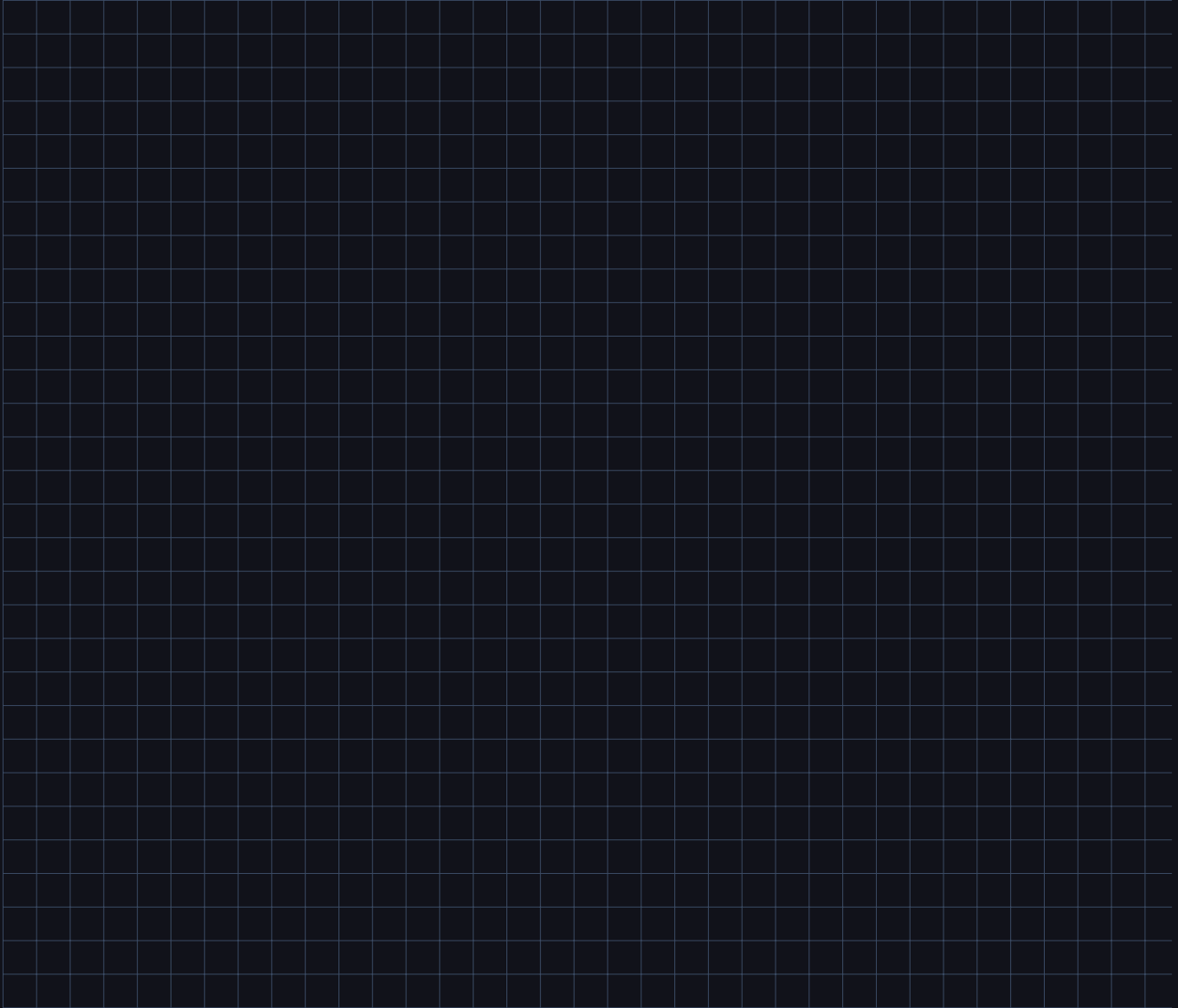
Simpson-Regel, $n = 2$; Stützstellen $-2, -1,5, -1$.

Randnotiz: Mehr Intervalle; zusammengesetzte Simpson-/Trapezregeln.



6.6 – Monte Carlo von Hand [Handrechnung]

Monte Carlo, $N = 4$, Stichproben $X_1 = -1,95$, $X_2 = -1,55$, $X_3 = -1,15$, $X_4 = -1,05$.



6.7 – Monte Carlo: Fehler & N erhöhen [Konzept]

Stichprobenvarianz und Standardfehler bestimmen. Was passiert bei $N = 16$? Warum ist der MC-Fehler dimensionsunabhängig?



6.8 – Simpson-Gewichte via Lagrange [Herleitung]

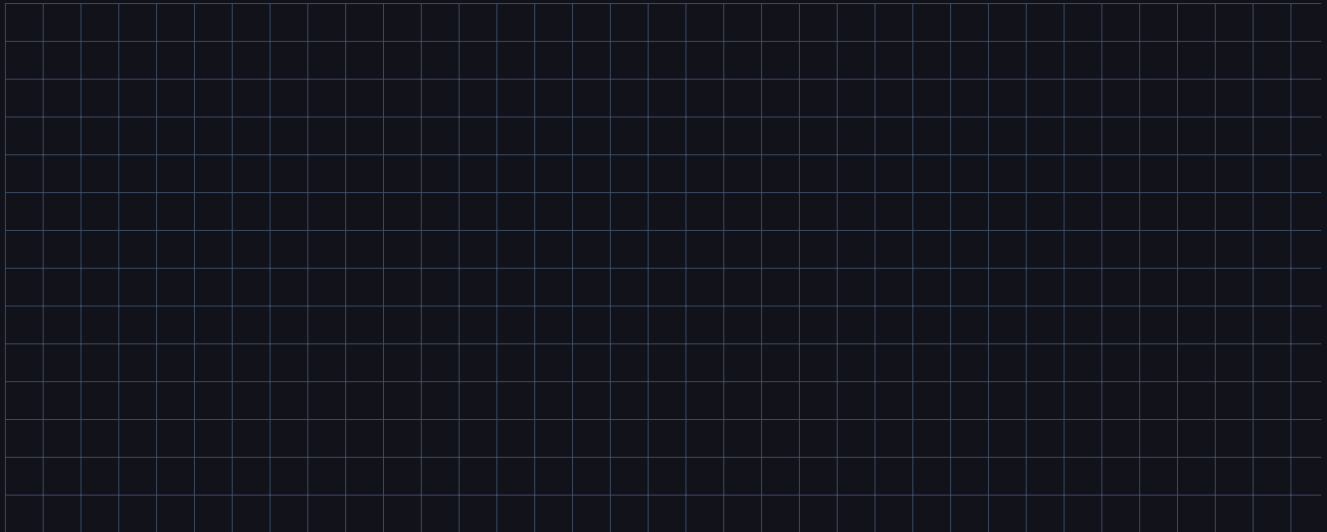
Leiten Sie die Simpson-Gewichte her, indem Sie die Lagrange-Basispolynome über $[a, b]$ integrieren.

Randnotiz: Beginnen Sie mit $x = 0$; was passiert, wenn Sie eine lineare Funktion quadratisch approximieren?



6.9 – Höhere Dimensionen (on your machine) [\[Code\]](#)

MC-Integration für ein d -dimensionales Integral (multivariate Gauß über Hyperwürfel); Fehler über d ; Versagen der Gitter-Quadratur.



Selbstreflexionsfragen

- Fehlervergleich Mittelpunkt/Trapez/Simpson/Monte Carlo?
- Warum Mittelpunkt besser als Links-/Rechts-Endpunkt?
- Zusammengesetzte Methoden: Genauigkeit & Trade-off?
- Warum versagt deterministische Quadratur in hohen Dimensionen?
- Mittelpunkt als Spezialfall von Monte Carlo?
- Intuition: warum “explodiert” MC in hohen Dim. nicht?
- Wann Simpson, wann Monte Carlo?
- Mini-Batch-Mittelwert in SGD und Monte Carlo?
- Wie hilft die Gradientenschätzung in SGD beim Entkommen?



Denkanstoß. Warum werden hochgradige Quadraturmethoden instabil, wenn sie Polynome niedrigen Grades approximieren?



7.1 – Loss bei Initialisierung [Handrechnung]

$\hat{y} = wx + b$, $L = \frac{1}{n} \sum_i (wx_i + b - y_i)^2$, Init $w = 0$, $b = \bar{y}$. Zeigen Sie $L(0, \bar{y}) = \text{Var}(y)$. Mit $\bar{y} = 231/14 = 16,5$, $\sum (y_i - \bar{y})^2 = 1401,5$: erster Loss-Wert? Bedeutung von 5000 bzw. 0,0001?



7.2 – Overfitting als Sanity-Check [Konzept]

Grad-4-Polynom durch zwei Punkte $((4,0;4), (9,5;21))$. Wie viele Null-Loss-Kurven gehen durch beide Punkte, und was sagt das über die Geometrie der Verlustlandschaft?



7.3 – Mini-Batch-SGD [Konzept]

In welchem Sinn ist Mini-Batch-SGD ein Monte-Carlo-Schätzer (was wird worüber gemittelt)? Ab welchem n kippt der Engpass, gegeben Fehler $\propto 1/\sqrt{B}$?



7.4 – Mehrere Random Seeds [Konzept]

$\hat{y} = w^2x + b$: warum ist der Ursprung ein Sattel, warum ist Null-Init eine Falle? Wozu Training aus mehreren Random Seeds?

7.5 – Quantisierung für Inferenz [Konzept]

Warum versendet man Modelle in float32, wenn int hier fast gleich gut war? Warum lässt sich hochpräzise trainiert niedrigpräzise einsetzen, während Training niedrigpräzise scheitert?

Selbstreflexionsfragen

- Warum entdeckt $\text{Loss} \rightarrow 0$ bei Überparametrisierung Pipeline-Bugs statt Optimierer-Bugs?
- Was sagt ein erster Loss von 5000 bzw. 0,0001 (statt ≈ 100)?
- Mini-Batch-SGD als Monte-Carlo-Schätzer – in welchem Sinn?
- $\hat{y} = w^2x + b$: Ursprung als Sattel, Null-Init als Falle?
- Hochpräzise trainieren, niedrigpräzise einsetzen – warum geht das?